

SALUS SECURITY

APR 2026



CODE SECURITY ASSESSMENT

ACH CHAIN

Overview

Project Summary

- Name: Alchemy Chain - contracts
- Platform: EVM-compatible chains
- Language: Solidity
- Audit Range: See [Appendix - 1](#)

Project Dashboard

Application Summary

Name	Alchemy Chain - contracts
Version	v2
Type	Solidity
Dates	May 24 2026
Logs	Apr 15 2026, May 24 2026

Vulnerability Summary

Total High-Severity issues	1
Total Medium-Severity issues	3
Total Low-Severity issues	1
Total informational issues	1
Total	6

Contact

E-mail: support@salusec.io

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

Content

Introduction	4
1.1 About SALUS	4
1.2 Audit Breakdown	4
1.3 Disclaimer	4
Findings	5
2.1 Summary of Findings	5
2.2 Notable Findings	6
1. Offchain server's native ether will be burned out by massive dust bridge	6
2. Missing pause function for bridge	7
3. Assets may be lost if sender is contract or abstract account	8
4. Centralization risk	9
5. Possible nonce conflict	10
2.3 Informational Findings	11
6. Implementation contract could be initialized by everyone	11
Appendix	12
Appendix 1 - Files in Scope	12

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (<https://t.me/salusec>), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Offchain bridge service's native ether will be burned out by massive dust bridge operations	High	Business Logic	Resolved
2	Missing pause function for bridge	Medium	Business Logic	Resolved
3	Assets may be lost if sender is contract or abstract account	Medium	Business Logic	Acknowledged
4	Centralization risk	Medium	Centralization	Acknowledged
5	Possible nonce conflict	Low	Business Logic	Resolved
6	Implementation contract could be initialized by everyone	Informational	Configuration	Acknowledged

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. Offchain bridge service's native ether will be burned out by massive dust bridge operations

Severity: High

Category: Business Logic

Target

- contracts/ACH.sol

Description

When users bridge ACH token from ACH chain to Ethereum, users will trigger the function `crossChainBridge`. The offchain bridge service will monitor the `CrossChainBridge` event and transfer the same amount on Ethereum.

Users don't need to pay bridge fees. Malicious users can bridge massive dust amounts ACH from ACH chain to Ethereum. Then offchain bridge service's native ether will be burned out quickly.

contracts/ACH.sol:L52-L55

```
function crossChainBridge(uint256 amount) external {  
    burn(msg.sender, amount);  
    emit CrossChainBridge(msg.sender, amount);  
}
```

Recommendation

Charge some bridge fees according to the current gas price.

Status

This issue has been resolved by the team.

2. Missing pause function for bridge

Severity: Medium

Category: Business Logic

Target

- contracts/ACH.sol

Description

When users bridge ACH token from ACH chain to Ethereum, users will trigger the function `crossChainBridge`. The offchain bridge service will monitor the `CrossChainBridge` event and transfer the same amount on Ethereum.

The offchain bridge service is implemented via one single process. If there is something wrong with the offchain bridge service, the bridge service will be paused. Users cannot receive the expected ACH on the destination chain.

The problem here is that the ACH contract misses one pause function for the bridge.

contracts/ACH.sol:L52-L55

```
function crossChainBridge(uint256 amount) external {  
    _burn(msg.sender, amount);  
    emit CrossChainBridge(msg.sender, amount);  
}
```

Recommendation

Add one pause feature for the on-chain bridge. If the offchain bridge service does not work for some reason, pause the on-chain bridge function.

Status

This issue has been resolved by the team.

3. Assets may be lost if sender is contract or abstract account

Severity: Medium

Category: Business Logic

Target

- contracts/ACH.sol

Description

When users bridge ACH token from ACH chain to Ethereum, users will trigger the function `crossChainBridge`. The offchain bridge service will monitor the `CrossChainBridge` event and transfer the same amount on Ethereum.

The offchain bridge service will always transfer or mint assets to the same address on the destination chain. If the sender is one contract address or abstract account, it's possible that the msg.sender does not own the same address on the destination chain. This may lead to asset loss.

contracts/ACH.sol:L52-L55

```
function crossChainBridge(uint256 amount) external {  
    burn(msg.sender, amount);  
    emit CrossChainBridge(msg.sender, amount);  
}
```

Recommendation

Add some input validation for the on-chain bridge function. For example, only EOA accounts can bridge assets from Ethereum to the ACH chain.

Status

This issue has been acknowledged by the team.

4. Centralization risk

Severity: Medium

Category: Centralization

Target

- contracts/ACH.sol

Description

In the ACH contract, there exists one privileged role, `owner`. This role has the authority to execute some key functions such as `adminTransfer`.

If this role's private key is compromised, an attacker could trigger these functions to withdraw all funds.

contracts/ACH.sol:L47-L49

```
function adminTransfer(address from, address to, uint256 amount) public onlyOwner {  
    _transfer(from, to, amount);  
}
```

Recommendation

We recommend transferring privileged accounts to multi-sig accounts with timelock governors for enhanced security. This ensures that no single person has full control over the accounts and that any changes must be authorized by multiple parties.

Status

This issue has been acknowledged by the team.

5. Possible nonce conflict

Severity: Low

Category: Business Logic

Target

- contracts/ACH.sol

Description

In the ACH contract, the owner role will take the responsibility to trigger admin transfer and mint ACH assets.

The offchain bridge service will mint ACH on the ACH chain via the owner's private key. And the owner may trigger `adminTransfer` via one wallet app, e.g. metamask. When the wallet app sends one transaction, they may choose one available nonce which may have already been used by the offchain bridge service.

contracts/ACH.sol:L47-L49

```
function adminTransfer(address from, address to, uint256 amount) public onlyOwner {  
    _transfer(from, to, amount);  
}
```

contracts/ACH.sol:L52-L55

```
function mint(address to, uint256 amount) public virtual onlyOwner {  
    _mint(to, amount);  
}
```

Recommendation

Choose two different roles, AdminRole and Bridge role for the function `adminTransfer` and `mint`.

Status

This issue has been resolved by the team.

2.3 Informational Findings

6. Implementation contract could be initialized by everyone

Severity: Informational

Category: Business Logic

Target:

- contracts/Permission.sol

Description

According to [OpenZeppelin](#), the implementation contract should not be left uninitialized.

An uninitialized implementation contract can be taken over by an attacker, which may impact the proxy. There is nothing preventing the attacker from calling the initialize function in Permission's implementation contract.

contracts/Permission.sol:L17-L19

```
function initialize() public initializer {  
    __Ownable_init(msg.sender);  
}
```

Recommendation

To prevent the implementation contract from being used, consider invoking the `_disableInitializers` function in the constructor of the Permission contract to automatically lock it when it is deployed.

Status

This issue has been acknowledged by the team.

Appendix

Appendix 1 - Files in Scope

This audit covered the following files:

File	SHA-1 hash
PermissionUpgradeableV2.sol	11107c7be71dc2df3b3b78d7ef99016fd2e6927f
GasPriceManagerV2.sol	65bd77648d6d86fe390c00da5efd135609342d77
ACH.sol	0c366c062a8ad7e390414915362f0bd4d1dda1fd
GasPriceManager.sol	9731ac2752edef6533f3d427860d1accc61ec786
Permission.sol	ecd79e629a77210b22fc9028828e76b8ba057f06
ACHV2.sol	a3b563bb25e3692645621671a3756c2aa41f6da2

And this audit covered the following fixed versions of the files:

File	SHA-1 hash
PermissionUpgradeableV2.sol	f8ede42e3b0129120cc95e10ddf2726e0e56a769
GasPriceManagerV2.sol	7192eee4be63a9004ad4c16cafa8f6968afc8460
ACH.sol	0c366c062a8ad7e390414915362f0bd4d1dda1fd
GasPriceManager.sol	9731ac2752edef6533f3d427860d1accc61ec786
Permission.sol	9bd6c49d36c6a91d83b53331007bcad4889d3622
ACHV2.sol	a3b563bb25e3692645621671a3756c2aa41f6da2